Lavinia Chiacchierini — teledidattica — 2099439

JZelda — diagramma delle classi

Model
Livello1
# inizializza(): void
NemicoFactory
+ creaNemico(livello, x, y): NemicoBase
«abstract»
Livello
# numeroLivello: int
# mappa: Tile[][]
# minion: List<NemicoBase>
# boss: Nemico
# rupie: List<Rupia>
# cristalli: List<Cristallo>
# inizializza(): void
- caricaMappa(): void
- riempiDiSabbia(): void
+ creaRupia(x, y, valore): void
+ creaCristallo(x, y): void
+ rimuoviOggettiRaccolti(): void
MostroTilapia
- VITA: int
- DANNO: int
- INTERVALLO_RAFFICA: long
- DURATA_ATTACCO: long
- DURATA_PAUSA: long
- frameTimer: int
- pinneAperte: boolean
- FRAME_PER_POSA: int
+ aggiorna(): void
Crociato
- VITA: int
- DANNO: int
- COOLDOWN: long
«enum»
GameState
+ MENU
+ GIOCO
+ SHOP
+ GAME_OVER
+ VITTORIA
crea
livelloCorrente 1
mappa 11x48
minion 4
cristalli *
rupie *
Cristallo
- x: int
- y: int
- larghezza: int
- altezza: int
- raccolto: boolean
+ applicaEffetto(link): void
+ raccogli(link): void
Rupia
- x: int
- y: int
- larghezza: int
- altezza: int
- valore: int
- raccolta: boolean
+ applicaEffetto(link): void
+ raccogli(link): void
NemicoBase
- xIniziale: int
- yIniziale: int
- RANGE_MOVIMENTO: int
- ciclo: Direzione[]
- faseCorrente: int
- ritorno: boolean
- random: Random
- contatoreFrame: int
- rupieRilasciate: int
- tempoDiMuoversi(): boolean
+ rimbalza(): void
+ muovi(): void
- raggiuntaLimite(dir): boolean
- tornatoAlCentro(dir): boolean
+ subisciDanno(danno): void
«abstract»
BossDistanza
# proiettili: List<Proiettile>
# cooldownAttacco: long
# ultimoAttacco: long
- intervalloRaffica: long
- durataAttacco: long
- durataPausa: long
- faseAttacco: boolean
- tempoInizioFase: long
+ muovi(): void
+ iniziaCombattimento(): void
+ spara(linkX, linkY): void
+ rimuoviProiettiliInattivi(): void
+ subisciDanno(danno): void
«abstract»
BossMischia
# xBase: int
# yBase: int
# RANGE_BOSS: int
# cooldownAttacco: long
# ultimoAttacco: long
- frameTimer: int
- fotogramma: int
- FRAME_PER_FOTOGRAMMA: int
- FOTOGRAMMI_CAMMINATA: int
- DURATA_POSA_ATTACCO: long
+ aggiorna(linkX, linkY): void
- avanzaCamminata(siStaMuovendo): void
+ muoviVersoLink(linkX, linkY): void
+ muovi(): void
+ puoAttaccare(): boolean
+ attacca(link): void
+ subisciDanno(danno): void
+ staAttaccando(): boolean
«enum»
Tile
+ SABBIA
+ ACQUA
+ PIETRE
+ MOSAICO
+ COLONNA
+ BIBLIOTECA
+ LIBRERIA
+ SOTTERRANEO
+ SCAFFALE
+ PRATO
+ FIORI
+ ALBERO
+ ROCCIA
+ LAVA
+ CASTELLO
+ ROVINE
applicaEffetto
applicaEffetto
Link
+ VITE: int
+ HP_PER_VITA: int
- vite: int
- spada: Spada
- scudo: Scudo
- bonusDanno: int
- rupie: int
- carote: int
- cristalli: int
- ultimoDanno: long
- COOLDOWN_DANNO: long
- scudoAttivo: boolean
- attaccando: boolean
- ultimoCristallo: long
- DURATA_POSA_CRISTALLO: long
+ muovi(): void
+ attacca(): void
+ terminaAttacco(): void
+ attivaScudo(): void
+ disattivaScudo(): void
+ subisciDanno(danno): void
+ subisciDannoProiettile(danno): void
+ equipaggiaSpada(spada): void
+ equipaggiaScudo(scudo): void
+ aumentaDanno(): void
+ aggiungiCristallo(): void
+ staMostrandoCristallo(): boolean
+ usaCarota(): void
+ aggiungiRupie(quantita): void
+ rimuoviRupie(quantita): void
+ aggiungiCarote(quantita): void
+ rimuoviCarota(): void
+ aggiungiVita(): void
+ reset(): void
+ riposiziona(): void
boss 1
«abstract»
Nemico
# vivo: boolean
+ muovi(): void
+ infliggiDanno(p): void
+ subisciDanno(danno): void
«interface»
SpawnaConiglietti
+ SPARPAGLIAMENTO: int
+ spawnaConiglietti(x, y): List<Coniglietti>
Coniglietti
- speciale: boolean
- attivo: boolean
- saltando: boolean
- contatoreSalto: int
- FRAME_SALTO: int
- FRAME_PAUSA: int
- fugaX: int
- fugaY: int
- fugaDecisa: boolean
- decidiFuga(linkX, linkY): void
- avanzaSalto(): boolean
+ muovi(): void
+ muoviLontanoDa(linkX, linkY): void
+ muoviVerso(linkX, linkY): void
+ interagisciConLink(link): void
+ disattiva(): void
coniglietti *
coniglietti *
ScudoBase
+ danniDaColpo(danno): int
+ danniDaProiettile(danno): int
proiettili *
Proiettile
- x: int
- y: int
- LARGHEZZA: int
- ALTEZZA: int
- direzione: Direzione
- danno: int
- attivo: boolean
+ muovi(): void
+ infliggiDanno(link): void
+ disattiva(): void
ciclo 4
crea
scudo 1
«abstract»
Scudo
# nome: String
+ danniDaColpo(danno): int
+ danniDaProiettile(danno): int
+ applicaEffetto(link): void
SpadaBase
+ applicaEffetto(link): void
SpadaFinale
+ applicaEffetto(link): void
«abstract»
Personaggio
# x: int
# y: int
# vita: int
# danno: int
# larghezza: int
# altezza: int
# direzione: Direzione
+ muovi(): void
+ subisciDanno(danno): void
Profilo
- filePath: Path
+ aggiungiProfilo(nickname, avatar): void
+ aggiornaStatistiche(nickname, vinta): void
notifica
legge
Shop
- itemDisponibili: List<Item>
+ PREZZO_CAROTA: int
+ acquista(link): boolean
Carote
+ applicaEffetto(link): void
spada 1
«abstract»
Spada
# danno: int
# nome: String
+ applicaEffetto(link): void
«enum»
Direzione
+ SU
+ GIU
+ SINISTRA
+ DESTRA
«enum»
Evento
+ VITA
+ VITE
+ SCUDO
+ SCUDO_EQUIPAGGIATO
+ SPADA
+ BONUS_DANNO
+ CRISTALLO
+ RUPIE
+ CAROTE
+ GAME_OVER
+ ATTACCO_LINK
+ DANNO_SUBITO
+ MORTE_NEMICO
+ MORTE_BOSS_MISCHIA
+ MORTE_BOSS_DISTANZA
+ ATTACCO_BOSS
+ VITA_BONUS
+ DANNO_CONIGLIO
+ CONIGLIO_DISATTIVATO
+ PROFILO
+ STATISTICHE
java standard
Observable
+ addObserver(o): void
# setChanged(): void
+ notifyObservers(arg): void
catalogo *
«interface»
Item
+ getNome(): String
+ applicaEffetto(link): void
ORDINE_RIGHE
Controller
JZelda
+ main(args): void
GameController
- FPS: int
- TARGET_TIME: long
- link: Link
- view: GameView
- profilo: Profilo
- inputHandler: InputHandler
- collisionController: CollisionController
- livelloCorrente: Livello
- numeroLivelloCorrente: int
- statoGioco: GameState
- coniglietti: List<Coniglietti>
- shop: Shop
- nicknameGiocatore: String
- gameThread: Thread
- schermataCorrente: int
- cameraX: int
- cameraXTarget: int
- VELOCITA_CAMERA: int
- AVANZAMENTO_LINK: int
- frameTransizione: int
- bossAttivato: boolean
- setupListeners(): void
+ iniziaPartita(): void
- caricaLivello(numero): void
- prossimoLivello(): void
- avviaGameLoop(): void
+ run(): void
- aggiornaGioco(): void
- inTransizione(): boolean
- avanzaTransizione(): void
- aggiornaCamera(): void
- bloccatoNellArenaDelBoss(dx): boolean
- gestisciInput(): void
- aggiornaNemici(): void
- aggiornaBoss(): void
- aggiornaConiglietti(): void
- verificaCompletamentoLivello(): void
+ update(o, arg): void
InputHandler
- tastiPremuti: Set<Integer>
+ keyPressed(e): void
+ keyReleased(e): void
+ keyTyped(e): void
+ azzera(): void
+ suPremuto(): boolean
+ giuPremuto(): boolean
+ sinistraPremuto(): boolean
+ destroPremuto(): boolean
+ attaccoPremuto(): boolean
+ scudoPremuto(): boolean
+ carotaPremuta(): boolean
AudioManager
- instance: AudioManager
- clipCorrente: Clip
- VOLUME_MUSICA: float
- VOLUME_EFFETTI: float
- impostaVolume(clip, decibel): void
+ play(filename): void
+ loop(filename): void
+ stopMusica(): void
CollisionController
- link: Link
- collidono(a, b): boolean
- collidonoProiettile(a, p): boolean
- linkStaColpendo(p): boolean
+ gestisciCollisioni(livello, coniglietti): void
- gestisciRaccoltaOggetti(livello): void
- gestisciCollisioniNemici(nemici): void
+ fuoriDallaMappa(p, dx, dy): boolean
+ bloccatoDaTile(p, dx, dy, mappa): boolean
+ chiSullaStrada(p, dx, dy, altriPersonaggi): Personaggio
- gestisciCollisioneBoss(boss, coniglietti): void
- gestisciCollisioniConiglietti(coniglietti): void
«interface»
KeyListener
java standard
View
GameView
- cardLayout: CardLayout
- contenitore: JPanel
- menuPanel: MenuPanel
- gamePanel: GamePanel
- shopPanel: ShopPanel
- profilePanel: ProfilePanel
- leaderboardPanel: LeaderboardPanel
- gameOverPanel: GameOverPanel
- vittoriaPanel: VittoriaPanel
+ mostraPanel(nome): void
+ update(o, arg): void
MappaProvvisoria
# caricaImmagini(): void
MenuPanel
- btnGioca: JButton
- btnProfilo: JButton
- btnClassifica: JButton
- btnEsci: JButton
- sfondo: BufferedImage
- PESO_SOPRA: double
+ mousePressed(e): void
- caricaSfondo(): BufferedImage
- nascondiPulsanti(): void
- mostraPulsanti(): void
# paintComponent(g): void
GameOverPanel
- btnRigioca: JButton
- btnMenu: JButton
- sfondo: BufferedImage
- PESO_SOPRA: double
- caricaSfondo(): BufferedImage
# paintComponent(g): void
VittoriaPanel
- btnMenu: JButton
- sfondo: BufferedImage
- PESO_SOPRA: double
- caricaSfondo(): BufferedImage
# paintComponent(g): void
ProfilePanel
- txtNickname: JTextField
- btnCreaProfilo: JButton
- listaProfili: JList<String>
- modelloLista: DefaultListModel<String>
- btnSeleziona: JButton
- btnIndietro: JButton
- profilo: Profilo
- sceltaAvatar: JComboBox<String>
- anteprimaAvatar: JLabel
- DIMENSIONE_ANTEPRIMA: int
- AVATAR_DISPONIBILI: String[][]
- mostraAvatarSelezionato(evento): void
- caricaAvatar(nomeFile): ImageIcon
+ aggiorna(): void
LeaderboardPanel
- modelloLista: DefaultListModel<String>
- listaClassifica: JList<String>
- btnIndietro: JButton
- profilo: Profilo
+ aggiorna(): void
ShopPanel
- lblRupie: JLabel
- btnAcquistaCarota: JButton
- btnContinua: JButton
- link: Link
+ aggiorna(): void
GamePanel
- RIGHE_FOGLIO: int
- GALLEGGIAMENTO: int
- PIXEL_PER_FOTOGRAMMA: int
- ORDINE_RIGHE: Direzione[]
- link: Link
- livello: Livello
- mappaView: MappaView
- coniglietti: List<Coniglietti>
- foglioLink: BufferedImage[][]
- imgLinkScudo: BufferedImage
- DIMENSIONE_ATTACCO_LINK: int
- imgLinkCristallo: BufferedImage
- ALTEZZA_POSA_CRISTALLO: int
- imgNemico: BufferedImage
- imgBoss: BufferedImage
- imgProiettile: BufferedImage
- imgRupia: BufferedImage
- imgCristallo: BufferedImage
- imgTilapia: BufferedImage[]
- imgVespasiano: BufferedImage
- imgGandalf: BufferedImage
- imgMorgana: BufferedImage
- imgSauron: BufferedImage
- imgLancia: BufferedImage[][]
- DIMENSIONE_LANCIA: int
- imgProiettileAcqua: BufferedImage[][]
- DIMENSIONE_PROIETTILE_ACQUA: int
- imgProiettileMagia: BufferedImage[][]
- DIMENSIONE_PROIETTILE_MAGIA: int
- imgProiettileFoglia: BufferedImage[][]
- DIMENSIONE_PROIETTILE_FOGLIA: int
- DIMENSIONE_ATTACCO_BOSS: int
- imgConiglietto: BufferedImage[][]
- minionPerLivello: List<EnumMap<Direzione, BufferedImage>>
- foglioBossMischia: Map<String, BufferedImage[][]>
- BOSS_DI_MISCHIA: String[]
- cameraX: int
- carica(nome): BufferedImage
- caricaSeEsiste(nome): BufferedImage
- caricaSprite(): void
- ritagliaFoglio(foglio): EnumMap<Direzione, BufferedImage>
- ritagliaGriglia(foglio, righe): BufferedImage[][]
- spriteMinion(direzione): BufferedImage
- disegnaSprite(g, img, fallback, x, y, w, h): void
- scegliMappa(numeroLivello): MappaView
# paintComponent(g): void
- disegnaBarra(g): void
- disegnaMappa(g): void
- disegnaLink(g): void
- fotogrammaLink(): BufferedImage
- disegnaOggetti(g): void
- disegnaNemici(g): void
- disegnaBoss(g): void
- fotogrammaBossMischia(boss): BufferedImage
- indiceRiga(direzione): int
- disegnaProiettili(g): void
- disegnaConiglietti(g): void
- fotogrammaConiglietto(c): BufferedImage
MappaLivello1
# caricaImmagini(): void
mappaView 1
«abstract»
MappaView
# immagini: EnumMap<Tile, BufferedImage>
- COLORE_DI_RISERVA: Color
# caricaImmagini(): void
# carica(nome): BufferedImage
+ disegna(g, mappa, cameraX): void
«interface»
Runnable
java standard
JFrame
java standard
«interface»
Observer
java standard
JPanel
java standard
Config
GameConstants
+ TILE_SIZE: int
+ SCHERMATA_TILE_W: int
+ SCHERMATA_TILE_H: int
+ SCHERMATA_W: int
+ SCHERMATA_H: int
+ NUM_SCHERMATE: int
+ NUM_LIVELLI: int
+ LIVELLI_CON_CRISTALLO: int
+ VELOCITA_LINK: int
+ VELOCITA_NEMICO: int
+ FRAME_PER_PASSO_NEMICO: int
+ RANGE_NEMICO: int
+ VELOCITA_CONIGLIO: int
+ DANNO_CONIGLIO: int
+ VELOCITA_PROIETTILE: int
+ MAPPA_TILE_W: int
+ MAPPA_TILE_H: int
+ MAPPA_W: int
+ MAPPA_H: int
+ FATTORE_CONVERSIONE: int
+ ALTEZZA_BARRA: int
+ TILE_SU_SCHERMO: int

Livello1 e' mostrato come rappresentante: Livello2 ... Livello8 hanno la stessa struttura e cambiano solo minion e boss.

Crociato rappresenta i boss di mischia: Barbero, LinkCattivo.

MostroTilapia rappresenta i boss a distanza: Vespasiano, Gandalf, Morgana, Sauron.

GameConstants e' usata da tutti i package; qui sono disegnate solo tre dipendenze rappresentative per leggibilita'.

MappaLivello1 e' mostrata come rappresentante: MappaLivello2 ... MappaLivello8 cambiano solo le tile associate alle immagini.